

Tensor.h & Tensor.c

Full Explanation — Concepts · Usage · Line-by-Line Code Walkthrough

Colour legend for code annotations:

	Function signature
	Variable declaration
	Logic / loop / condition
	Return statement
	Guard / error check
	Comment

- 1 — Technical Concepts Glossary
- 2 — Tensor.h — Every Struct Explained
- 3 — calcNumberOfElementsByShape / ByTensor / ByParameter
- 4 — calcBytesPerElement / calcBitsPerElement
- 5 — calcBytesPerTensor / calcBitsPerTensor
- 6 — calcNumberOfBytesForData
- 7 — tensorBoolGet
- 8 — tensorBoolSet
- 9 — getBitmask / readByte / writeByte
- 10 — byteConversion
- 11 — getParamFromParameter / getGradFromParameter
- 12 — setOrderOfDimsForNewTensor / setShape / setParameterValues
- 13 — setTensorValues / setTensorValuesForConversion
- 14 — transposeTensor
- 15 — printTensor / printShape
- 16 — copyData / copyShape / copyQuantization / copyTensor
- 17 — Complete Usage Workflow
- 18 — Common Mistakes
- 19 — Quick Reference Card

1 — Technical Concepts Glossary

struct: A container that groups related variables under one name.

Analogy: A passport — groups name, date of birth, and photo in one document.

typedef: Gives an existing type a shorter alias. `typedef struct Foo {} foo_t;` lets you write `foo_t` instead of `struct Foo`.

Analogy: Calling 'mobile telephone' just 'phone'.

pointer (*): A variable that stores a memory address. It tells you WHERE data lives, not what the data is.

Analogy: A sticky note with a room number — it points to the room, it is not the room.

NULL pointer: A pointer that points to nothing (address 0). Means 'not set' or 'not active'.

Analogy: A blank sticky note — points nowhere.

uint8_t: Unsigned 8-bit integer. Range 0–255. One byte. All tensor raw data is stored as `uint8_t[]`.

Analogy: A single box that holds a number 0–255.

size_t: Unsigned integer sized to the platform: 4 bytes on STM32, 8 bytes on 64-bit PC. Used for counts and indices.

Analogy: A ruler that adapts its length to the country.

enum: A list of named integer constants. `INT32=0`, `FLOAT32=1` etc. You use the name; C stores the number.

Analogy: A menu — you order by name, the kitchen uses a number.

void *: Pointer to any type. The compiler does not know what is at that address. Cast it before use.

Analogy: A USB port — accepts anything, but the device needs its own driver.

switch / case: A multi-way branch. `switch(x) { case A: ...; break; case B: ...; break; }` jumps to the matching case.

Analogy: A train junction — the train goes to whichever track matches the signal.

memcpy(dest, src, n): Standard C: copies exactly `n` bytes from `src` to `dest`. Does not know the type — raw bytes.

Analogy: A photocopier — byte-for-byte duplicate.

sizeof(type): A compile-time operator that returns the byte size of a type. `sizeof(float)=4`, `sizeof(int32_t)=4`.

Analogy: Asking 'how many pages is this form?' before printing.

ceilf(x): Standard math: rounds a float UP to the nearest integer. `ceilf(1.25)=2`, `ceilf(1.0)=1`.

Analogy: Always rounding up to the next whole floor in a building.

bitwise AND &: Compares bit by bit: result bit = 1 only if BOTH bits are 1. Used to isolate or clear bits.

Analogy: Two people must both say YES.

bitwise OR |: Result bit = 1 if EITHER bit is 1. Used to set a specific bit.

Analogy: Either person saying YES is enough.

bitwise NOT ~: Flips every bit: 0→1, 1→0. Used to create an inverted mask.

Analogy: Turning off all lights that are on and turning on all that are off.

left shift <<: Moves all bits left by `N` positions (multiplies by 2^N). `1<<3 = 8`.

Analogy: Adding `N` zeros to the right of a binary number.

right shift >>: Moves all bits right by N positions (divides by 2^N). $8 \gg 2 = 2$.

Analogy: Removing N digits from the right of a binary number.

|= and &=: Compound assignment operators. $x |= \text{mask}$ means $x = x | \text{mask}$. $x \&= \text{mask}$ means $x = x \& \text{mask}$.

Analogy: Like += but for bitwise operations.

deep copy vs shallow copy: Deep: all data duplicated, fully independent. Shallow: only pointers copied, both still share the same data.

Analogy: Deep=photocopying a book. Shallow=giving someone the book's location.

exit(1): Immediately stops the entire program with error code 1. Used when something is fatally wrong.

Analogy: Pulling the fire alarm — everything stops.

2 — Tensor.h — Every Struct Explained

tensorStorageId

```
typedef void *tensorStorageId;
```

Creates a new type name 'tensorStorageId' that means 'void* — pointer to anything'. Used as an opaque storage identifier (like a locker number) for future storage backends.

shape_t

```
typedef struct shape {
```

A new struct type called 'shape'. Holds all dimension information.

```
size_t numberOfDimensions;
```

How many dimensions: 1=vector, 2=matrix, 3=cube, etc.

```
size_t *dimensions;
```

Pointer to an array. Each slot holds the size of one dimension. [3,4] means 3 rows and 4 columns.

```
size_t *orderOfDimensions;
```

Pointer to an array that defines traversal order. Default [0,1,2]. After transpose it changes to e.g. [1,0] — no data moves.

```
} shape_t;
```

Closes the struct and gives it the alias shape_t.

sparsity_t

```
typedef enum {
```

Defines an enumeration — a list of named integer constants.

```
SPARSITY_TYPE_1,
```

First sparsity scheme (value = 0). Placeholder for future use.

```
SPARSITY_TYPE_2
```

Second sparsity scheme (value = 1). Placeholder.

```
} sparsityType_t;
```

Closes the enum with alias sparsityType_t.

```
typedef struct sparsityConfig {
```

Empty struct — a placeholder for future sparsity configuration fields.

```
} sparsityConfig;
```

Closes the empty config struct.

```
typedef struct sparsity {
```

The actual sparsity descriptor attached to a tensor.

```
sparsityType_t type;
```

Which sparsity scheme is used.

```
sparsityConfig *config;
```

Pointer to config details (currently empty). NULL = no config.

```
} sparsity_t;
```

Closes the struct with alias sparsity_t.

■ If tensor->sparsity == NULL: the tensor is DENSE (all values stored). If non-NULL: the tensor is SPARSE (some values are zero and not stored).

tensor_t — Main Data Container

```
typedef struct tensor {
```

The central struct — represents one tensor (weights, gradient, etc.).

```
uint8_t *data;
```

Pointer to the raw byte buffer. This is where all the actual numbers live. Every other field is metadata ABOUT this buffer.

```
shape_t *shape;
```

Pointer to a shape_t struct. Tells us how many dimensions and their sizes.

<code>quantization_t *quantization;</code>	Pointer to quantization info: the number format (FLOAT32, SYM_INT32 etc.) and the scale/zeroPoint.
<code>sparsity_t *sparsity;</code>	Pointer to sparsity info. NULL = dense tensor.
<code>} tensor_t;</code>	Closes the struct with alias <code>tensor_t</code> . Total size on STM32: 4 pointers × 4 bytes = 16 bytes.

★ **All fields are pointers** — `tensor_t` itself is only 16 bytes. Passing `tensor_t*` to any function costs just 4 bytes. Multiple tensors can share the same `shape_t` (e.g. weight + gradient).

parameter_t — Weight + Gradient Pair

<code>typedef struct parameter {</code>	Groups a weight tensor and its gradient tensor.
<code>tensor_t *param;</code>	The learnable weight tensor — the actual values being trained.
<code>tensor_t *grad;</code>	The gradient tensor — stores <code>dLoss/dParam</code> . Same shape as <code>param</code> . Updated each backprop pass.
<code>} parameter_t;</code>	Closes the struct with alias <code>parameter_t</code> .

3 — Element Counting Functions

calcNumberOfElementsByShape

WHAT IT DOES	Multiplies all dimension sizes together to get the total element count.
WHY IT EXISTS	Every other function (size calc, loops, allocation) needs the element count first. This is called everywhere.
HOW TO USE IT	Pass a shape_* to get the total number of values stored in a tensor of that shape.
<pre>size_t calcNumberOfElementsByShape(shape_t *shape) { size_t numElem = 1; size_t numberOfDimensions = shape->numberOfDimensions; size_t *dimensions = shape->dimensions; for (size_t i = 0; i < numberOfDimensions; i++) { numElem *= dimensions[i]; } return numElem; }</pre>	Takes a pointer to a shape_t. Returns size_t (always non-negative). Start the product at 1 (not 0 — multiplying by 0 would give 0 always). Copy the field into a local variable. Avoids repeated pointer dereferences in the loop. Local pointer to the dimensions array. Avoids writing shape->dimensions[i] each iteration. Loop over every dimension. i goes from 0 to (numberOfDimensions - 1). Multiply running product by the size of dimension i. After all iterations: numElem = d0 * d1 * ... * d(n-1). Return the total element count.

```
// Example: shape [3, 4, 2]
// i=0: numElem = 1 * 3 = 3
// i=1: numElem = 3 * 4 = 12
// i=2: numElem = 12 * 2 = 24
// returns 24
```

calcNumberOfElementsByTensor / ByParameter

<pre>size_t calcNumberOfElementsByTensor(tensor_t *tensor) { return calcNumberOfElementsByShape(tensor->shape); }</pre>	Takes a tensor_*. Just extracts the shape pointer and delegates. tensor->shape dereferences tensor to get the shape pointer. Then calls the main function. One-line wrapper.
<pre>size_t calcNumberOfElementsByParameter(parameter_t *parameter) { return calcNumberOfElementsByShape(parameter->para m->shape); }</pre>	Takes a parameter_*. Uses param (weight tensor) — grad always has the same shape. parameter->param gets the weight tensor pointer. ->shape gets its shape pointer. Then calls the main function.

4 — calcBytesPerElement / calcBitsPerElement

calcBytesPerElement

WHAT IT DOES	Returns how many bytes ONE element occupies based on the quantization type.
WHY IT EXISTS	Byte-level pointer arithmetic and memcpy need to know the element size. Centralises the type→size mapping.
HOW TO USE IT	Call before allocating memory or advancing a pointer by one element. Note: misleading for BOOL (returns 1, but BOOL uses 1 bit, not 1 byte).

<code>size_t calcBytesPerElement(quantization_t *quantization) {</code>	Takes the quantization struct pointer. Returns a byte count.
<code>switch (quantization->type) {</code>	Branch on the type tag. type is a qtype_t enum: INT32, FLOAT32, SYM_INT32, SYM, ASYM, BOOL.
<code>case INT32:</code>	If type is INT32 (raw 32-bit integers, no quantization).
<code>return sizeof(int32_t);</code>	sizeof(int32_t) = 4. This is a compile-time constant — the compiler replaces it with 4.
<code>case FLOAT32:</code>	If type is FLOAT32 (standard 32-bit floating point).
<code>return sizeof(float);</code>	sizeof(float) = 4 on both STM32 and PC.
<code>case SYM_INT32:</code>	Symmetric quantization stored as 32-bit integers.
<code>return sizeof(int32_t);</code>	Same as INT32 — data is stored as int32_t, scale is metadata.
<code>case SYM: {</code>	Symmetric quantization with variable bit-width (e.g. 4-bit or 8-bit).
<code>symQConfig_t *symQC = quantization->qConfig;</code>	Cast the void* qConfig to the correct config type: symQConfig_t*. This gives access to the qBits field.
<code>return (size_t)ceilf((float)symQC->qBits / 8.0f);</code>	qBits could be 4, 8, 12, etc. Divide by 8 to get bytes. ceilf rounds UP: 4/8=0.5→1 byte, 12/8=1.5→2 bytes.
<code>}</code>	
<code>case ASYM: {</code>	Asymmetric quantization (has a zero-point offset).
<code>asymQConfig_t *asymQConfig = quantization->qConfig;</code>	Cast qConfig to asymQConfig_t* to access qBits.
<code>uint32_t qBits = asymQConfig->qBits;</code>	Extract qBits into a local variable for clarity.
<code>return ceil((float)qBits / (float)8);</code>	Same ceiling-division as SYM case.
<code>}</code>	
<code>case BOOL:</code>	Boolean tensor — 1 bit per value (8 packed per byte).
<code>return 1;</code>	Returns 1. WARNING: this is misleading — BOOL actually uses 1 BIT, not 1 BYTE. Use calcNumberOfBytesForData for correct BOOL allocation.
<code>default:</code>	Unknown type — should never happen.
<code>PRINT_ERROR("Unknown QType!");</code>	Print error via the debug macro from Common.h.
<code>exit(1);</code>	Stop the program immediately. Returning garbage would silently corrupt memory.
<code>}</code>	
<code>}</code>	

calcBitsPerElement

WHAT IT DOES	Returns how many BITS one element occupies. Accurate for sub-byte formats.
WHY IT EXISTS	calcBytesPerElement loses precision for 4-bit or 1-bit types. Bits is the correct unit.
HOW TO USE IT	Used internally by byteConversion and calcNumberOfBytesForData.

size_t calcBitsPerElement(quantization_t *quantization) {	
switch (quantization->type) {	Same switch structure as calcBytesPerElement.
case INT32:	
return sizeof(int32_t) * 8;	sizeof(int32_t)=4 bytes. 4*8=32 bits.
case FLOAT32:	
return sizeof(float) * 8;	sizeof(float)=4. 4*8=32 bits.
case SYM_INT32:	
return sizeof(int32_t) * 8;	32 bits.
case SYM: {	
symQConfig_t *symQC = quantization->qConfig;	Cast qConfig to get access to qBits.
return symQC->qBits;	Return the exact bit width — 4, 8, 12, etc. No rounding needed here.
}	
case ASYM: {	
asymQConfig_t *asymQConfig = quantization->qConfig;	
return asymQConfig->qBits;	Same — return exact bit width.
}	
case BOOL:	
return 1;	Exactly 1 bit per boolean value.
default:	
PRINT_ERROR("Unknown QType!");	
exit(1);	Stop on unknown type.
}	
}	

5 — calcBytesPerTensor / calcBitsPerTensor

calcBitsPerTensor

<code>size_t calcBitsPerTensor(tensor_t *tensor) {</code>	Takes a whole tensor. Returns total bit count.
<code>size_t bitsPerElement = calcBitsPerElement(tensor->quantization);</code>	Ask: how many bits does ONE element need for this tensor's type?
<code>size_t numElements = calcNumberOfElementsByShape(tensor->shape);</code>	Ask: how many elements does this tensor have?
<code>return bitsPerElement * numElements;</code>	Total bits = bits per element × number of elements. Simple multiplication.
<code>}</code>	

calcBytesPerTensor

WHAT IT DOES	Returns total byte size of a tensor's data buffer.
WHY IT EXISTS	Convenience function — combines calcBitsPerTensor and division by 8.
HOW TO USE IT	Call when you need the total byte size of a tensor you already have.

<code>size_t calcBytesPerTensor(tensor_t *tensor) {</code>	Takes a tensor, returns total bytes.
<code>size_t bitsPerTensor = calcBitsPerTensor(tensor);</code>	Get total bits via the function above.
<code>return bitsPerTensor / 8;</code>	Convert bits to bytes by dividing by 8. Note: this divides integers, so it truncates. Use calcNumberOfBytesForData for precise packed-format sizes.
<code>}</code>	

6 — calcNumberOfBytesForData

WHAT IT DOES	Returns exact total bytes for N elements of the given quantization type. Handles BOOL packing correctly with ceiling division.
WHY IT EXISTS	This is THE function to call before allocating a data buffer. calcBytesPerTensor can truncate for packed formats; this one is always exact.
HOW TO USE IT	Call with (tensor->quantization, n) to get the correct byte count for reserveMemory().

<code>size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements) {</code>	Takes quantization type info and element count. Returns exact byte count.
<code>switch (q->type) {</code>	Branch on quantization type.
<code>case FLOAT32:</code>	Standard 32-bit float.
<code>return numberOfElements * sizeof(float);</code>	4 bytes per element. 12 floats = 48 bytes.
<code>case INT32:</code>	Raw 32-bit integer.
<code>return numberOfElements * sizeof(int32_t);</code>	4 bytes per element.
<code>case SYM_INT32:</code>	Symmetric quantization stored as int32.
<code>return numberOfElements * sizeof(int32_t);</code>	4 bytes per element.
<code>case SYM:</code>	Symmetric quantization with variable bit-width.

<code>return (calcBitsPerElement(q) * numberOfElements + 7) / 8;</code>	bits × n gives total bits. (+7)/8 is ceiling division to bytes. Example: 4-bit × 9 values = 36 bits → (36+7)/8 = 5 bytes.
<code>case ASYM: {</code>	Asymmetric quantization.
<code>size_t bitsPerElement = calcBitsPerElement(q);</code>	Get the bit width for this type.
<code>return (bitsPerElement * numberOfElements + 7) / 8;</code>	Same ceiling-division formula as SYM.
<code>}</code>	
<code>case BOOL:</code>	Boolean — 1 bit per value, 8 per byte.
<code>return (numberOfElements + 7) / 8;</code>	Special case of ceiling division: (n+7)/8. 9 booleans → (9+7)/8 = 2 bytes. 8 booleans → (8+7)/8 = 1 byte.
<code>default:</code>	
<code>PRINT_ERROR("Unknown QType!");</code>	
<code>exit(1);</code>	Stop on unknown type.
<code>}</code>	
<code>}</code>	

★ **The (n+7)/8 ceiling trick:** Integer / always truncates. $9/8 = 1$ (WRONG). $(9+7)/8 = 16/8 = 2$ (CORRECT). General: $\text{ceil}(a/b) = (a + b - 1) / b$. For bytes: $(\text{bits} + 7) / 8$.

7 — tensorBoolGet

WHAT IT DOES	Reads one boolean bit from a packed BOOL tensor at position flatIndex.
WHY IT EXISTS	BOOL tensors pack 8 values per byte. data[flatIndex] gives the wrong byte. This function does the byteIndex/bitIndex math correctly.
HOW TO USE IT	Use whenever you need to read one bit from a sparsity mask. Never access data[] directly on a BOOL tensor.

bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex) {	tensor is const — this function promises not to modify it. flatIndex is the linear position of the bool value (0, 1, 2, ...).
if (tensor->quantization->type != BOOL) { PRINT_ERROR("tensorBoolGet called on non-BOOL tensor"); exit(1); }	Guard check: is this really a BOOL tensor? If not, we would return garbage from float/int data. Print an error message using the Common.h macro. Stop the program. Silently returning garbage would be worse.
size_t byteIndex = flatIndex / 8; size_t bitIndex = flatIndex % 8;	Integer division: which byte contains this bit? Index 10 → byte 1 (bytes hold indices 0-7, 8-15, ...). Remainder: which bit WITHIN that byte (0=rightmost, 7=leftmost)? Index 10 → bit 2 within byte 1.
return ((tensor->data[byteIndex] >> bitIndex) & 1u) != 0;	Three operations in one line: 1. data[byteIndex] — fetch the byte containing our bit. 2. >> bitIndex — shift right so our target bit moves to position 0. 3. & 1u — mask off all other bits; result is 0 or 1. 4. != 0 — convert to bool (true/false).
}	

```
// Example: flatIndex=10
// byteIndex = 10/8 = 1 → look in data[1]
// bitIndex = 10%8 = 2 → look at bit position 2
//
// data[1] = 0b10110100
// >> 2 = 0b00101101 ← bit 2 moved to position 0
// & 1u = 0b00000001 = 1
// != 0 = true ← weight 10 is active
```

8 — tensorBoolSet

WHAT IT DOES	Writes one bit into a packed BOOL tensor at position flatIndex. Sets it to 1 (true) or 0 (false) without touching any other bit.
WHY IT EXISTS	RigL grow/drop steps activate and prune individual weights. Each weight is one bit in the mask.
HOW TO USE IT	tensorBoolSet(mask, idx, true) to activate weight idx. tensorBoolSet(mask, idx, false) to prune it.

void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value) {	tensor is non-const — we will modify it. value is the bool to write: true=1, false=0.
if (tensor->quantization->type != BOOL) { PRINT_ERROR("tensorBoolSet called on non-BOOL tensor"); exit(1); }	Guard check — same as tensorBoolGet. Stop if wrong type.
size_t byteIndex = flatIndex / 8; size_t bitIndex = flatIndex % 8;	Which byte to modify? Same formula as tensorBoolGet. Which bit within that byte?
if (value) { tensor->data[byteIndex] = (uint8_t)(1u << bitIndex); } else { tensor->data[byteIndex] &= (uint8_t)~(1u << bitIndex); }	If we want to SET the bit to 1: 1u << bitIndex creates a mask with exactly one 1 at bitIndex. Example: bitIndex=2 → 0b00000100. = (OR-assign) forces that bit to 1, leaves all others unchanged. If we want to CLEAR the bit to 0: 1u << bitIndex → 0b00000100. ~(...) inverts it → 0b11111011. &= (AND-assign) forces only the target bit to 0, leaves all others unchanged.
}	

```
// SET bit 2 to 1:
// 1u << 2 = 0b00000100
// existing byte = 0b10110000
// existing |= mask = 0b10110100 ← bit 2 is now 1

// CLEAR bit 2 to 0:
// 1u << 2 = 0b00000100
// ~(1u<<2) = 0b11111011
// existing = 0b10110100
// existing &= ~mask= 0b10110000 ← bit 2 is now 0
```

9 — getBitmask / readByte / writeByte

■ These three functions are internal helpers for byteConversion. You will not call them directly in user code.

getBitmask

WHAT IT DOES	Returns a byte with 1s in bit positions [startbit, endbit) and 0s elsewhere.
WHY IT EXISTS	readByte and writeByte need a mask to isolate or protect specific bits. getBitmask builds that mask from start/end positions.
HOW TO USE IT	Called internally by readByte and writeByte.

<pre>uint32_t getBitmask(uint32_t startbit, uint32_t endbit) {</pre>	Takes global bit positions, returns a byte mask.
<pre> uint32_t endbitInternal = endbit - (startbit / 8) * 8;</pre>	Normalise endbit to be relative to the start of the byte. startbit/8 gives the byte index; *8 gets the byte's starting bit. Subtracting gives the position within the byte.
<pre> uint32_t startbitInternal = startbit - (startbit / 8) * 8;</pre>	Same normalisation for startbit. Result: both are now 0–7.
<pre> uint32_t counter = 0;</pre>	Will accumulate the mask value by summing powers of 2.
<pre> uint32_t value = 1;</pre>	value = 2 ⁱ at each step: 1, 2, 4, 8, 16, 32, 64, 128.
<pre> for (size_t i = 0; i < 8; i++) {</pre>	Loop over all 8 bit positions in a byte.
<pre> if ((i >= startbitInternal) & (endbitInternal > i)) {</pre>	Is bit position i inside the range [startbit, endbit)? Note: & not && — always evaluates both conditions (no short-circuit). Acceptable here because both are simple comparisons.
<pre> counter += value;</pre>	Add this bit's value to the mask. Equivalent to setting bit i in the result.
<pre> }</pre>	
<pre> value *= 2;</pre>	Next power of 2: 1→2→4→8→...→128.
<pre> }</pre>	
<pre> return counter;</pre>	counter is now a byte with 1s exactly at positions [startbit, endbit).
<pre>}</pre>	

```
// getBitmask(2, 5) → bits 2, 3, 4 should be 1
// i=0: 0 < 2 → skip. i=1: 1 < 2 → skip.
// i=2: 2 >= 2 AND 5 > 2 → counter += 4 (counter=4)
// i=3: 3 >= 2 AND 5 > 3 → counter += 8 (counter=12)
// i=4: 4 >= 2 AND 5 > 4 → counter += 16 (counter=28)
// i=5: 5 >= 2 AND 5 > 5 = false → skip
// counter = 28 = 0b00011100
```

readByte

<pre>uint8_t readByte(uint8_t data, uint8_t startbit, uint8_t endbit) {</pre>	Extracts bits [startbit, endbit) from data and returns them right-aligned.
<pre> uint8_t bitmask = getBitmask(startbit, endbit);</pre>	Get a mask with 1s at the bit positions we want to read.

<code>uint8_t intermediate = data & bitmask;</code>	AND with mask: zeros out all bits outside [startbit, endbit). Only the target bits survive.
<code>intermediate >>= startbit - (startbit / 8) * 8;</code>	Right-shift by the within-byte startbit. This moves the extracted bits to start at position 0.
<code>return intermediate;</code>	Return the extracted value, right-aligned.
<code>}</code>	

writeByte

<code>uint8_t writeByte(uint8_t existingData, uint8_t data,</code>	existingData: the byte we are writing INTO. data: the bits to insert.
<code>uint8_t startbit, uint8_t endbit) {</code>	startbit/endbit: where in the byte to insert.
<code>uint8_t startbitInternal = startbit - (startbit / 8) * 8;</code>	Normalise startbit to within-byte position (0–7).
<code>uint8_t endbitInternal = endbit - (startbit / 8) * 8;</code>	Normalise endbit the same way.
<code>uint8_t bitmask = getBitmask(startbitInternal, endbitInternal);</code>	Mask that covers exactly the target bit range.
<code>data <<= startbitInternal;</code>	Shift data left to align it with the target position in the byte.
<code>uint8_t intermediate = data & bitmask;</code>	Mask off any overflow — ensures data does not bleed into adjacent bits.
<code>existingData = intermediate existingData;</code>	OR merges the new bits into the existing byte. Bits outside the target range are unchanged.
<code>return existingData;</code>	Return the byte with the new bits inserted.
<code>}</code>	

10 — byteConversion

WHAT IT DOES	Converts a packed array of numValues values from dataInBits-wide to dataOutBits-wide format. Handles cross-byte boundaries automatically.
WHY IT EXISTS	Quantization conversion (FLOAT32→INT8, INT32→INT4) requires repacking bits. This function handles any bit-width combination.
HOW TO USE IT	Called internally by conversion functions — not in user code directly.

<pre>void byteConversion(uint8_t *dataIn, size_t dataInBits, uint8_t *dataOut, size_t dataOutBits, size_t numValues) {</pre>	numValues: how many values to convert. Not bytes — values.
<pre>memset(dataOut, 0, (numValues * dataOutBits - 1) / 8 + 1);</pre>	Zero-fill the entire output buffer first. This is important because writeByte uses OR — it only sets bits, never clears them. Starting from zero ensures no stale bits remain.
<pre>size_t dataOutIndex = 0; size_t dataInIndex = 0; int dataOutStartbit = 0; int dataInStartbit = 0; int dataInEndbit = (int)dataInBits; int dataOutEndbit = (int)dataOutBits;</pre>	Which byte in the output buffer we are currently writing. Which byte in the input buffer we are currently reading. Cursor: current bit position within the current output byte. Cursor: current bit position within the current input byte. End of first input value (relative to start of the value). End of first output value.
<pre>for (size_t i = 0; i < numValues; i++) { while ((dataInStartbit < dataInEndbit) </pre>	Outer loop: process each value.
<pre>(dataOutStartbit < dataOutEndbit)) {</pre>	Inner loop: keep going while either the input or output value is not fully processed.
<pre>uint8_t data = 0;</pre>	Bits to transfer this iteration.
<pre>if (dataInStartbit < dataInEndbit) { data = readByte(dataIn[dataInIndex], dataInStartbit, dataInEndbit); }</pre>	Guard: only read from input if there are still bits to read. Extract bits from the current input byte starting at dataInStartbit.
<pre>if (dataOutStartbit < dataOutEndbit) { dataOut[dataOutIndex] =</pre>	Guard: only write to output if there are still bits to write.
<pre>writeByte(dataOut[dataOutIndex], data, dataOutStartbit, dataOutEndbit); }</pre>	Write the extracted bits into the output byte at dataOutStartbit.
<pre>int valuesRead = min(dataInEndbit - dataInStartbit, 8 - dataInStartbit % 8); int valuesWritten = min(dataOutEndbit - dataOutStartbit, 8 - dataOutStartbit % 8); int minValue = min(valuesRead, valuesWritten);</pre>	How many bits CAN we read this iteration? Limited by: (a) bits left in value, (b) bits left in byte. How many bits CAN we write this iteration? Same logic for output. Transfer only as many bits as both sides can handle simultaneously.

<code>uint8_t deltaIn = minValue;</code>	Advance input cursor by this many bits.
<code>uint8_t deltaOut = minValue;</code>	Advance output cursor by this many bits.
<code>if (dataInStartbit == dataInEndbit) {</code>	Edge case: input is exhausted (widening — output element is larger). Advance output by a full byte-worth.
<code>dataOutStartbit += valuesWritten;</code>	
<code>deltaOut = valuesWritten;</code>	
<code>} else {</code>	
<code>dataOutStartbit += minValue;</code>	Normal case: advance output cursor by the shared minimum.
<code>}</code>	
<code>if (dataOutStartbit == dataOutEndbit) {</code>	Edge case: output is full (narrowing — input element is larger). Advance input by a full byte-worth.
<code>dataInStartbit += valuesRead;</code>	
<code>deltaIn = valuesRead;</code>	
<code>} else {</code>	
<code>dataInStartbit += minValue;</code>	Normal case: advance input cursor.
<code>}</code>	
<code>if (dataInStartbit / 8 > (dataInStartbit - deltaIn) / 8) {</code>	Did the input cursor just cross a byte boundary? If so, move to next input byte.
<code>dataInIndex += 1;</code>	
<code>}</code>	
<code>if (dataOutStartbit / 8 > (dataOutStartbit - deltaOut) / 8) {</code>	Did the output cursor just cross a byte boundary? If so, move to next output byte.
<code>dataOutIndex += 1;</code>	
<code>}</code>	
<code>}</code>	End of inner while-loop.
<code>dataInStartbit = dataInEndbit % 8;</code>	Reset input cursor to the bit position after the last value (could be mid-byte).
<code>dataInEndbit = dataInStartbit + dataInBits;</code>	Set end of next input value.
<code>dataOutStartbit = dataOutEndbit % 8;</code>	Reset output cursor.
<code>dataOutEndbit = dataOutStartbit + dataOutBits;</code>	Set end of next output value.
<code>}</code>	End outer for-loop.
<code>}</code>	

11 — getParamFromParameter / getGradFromParameter

```
tensor_t *getParamFromParameter(parameter_t  
*parameter) {
```

```
    return parameter->param;
```

```
}
```

Takes a parameter_t*. Returns the weight tensor pointer.

parameter->param dereferences the pointer and returns the param field. One-line getter — exists for readability.

```
tensor_t *getGradFromParameter(parameter_t  
*parameter) {
```

```
    return parameter->grad;
```

```
}
```

Takes a parameter_t*. Returns the gradient tensor pointer.

Same — returns the grad field.

```
// Usage:
```

```
tensor_t *w = getParamFromParameter(myParam);
```

```
tensor_t *g = getGradFromParameter(myParam);
```

```
// Same as: myParam->param and myParam->grad
```

```
// But clearer and easier to search in code
```

12 — setOrderOfDimsForNewTensor / setShape / setParameterValues

setOrderOfDimsForNewTensor

```
void setOrderOfDimsForNewTensor(size_t
numberOfDimensions, size_t *orderOfDimensions) {
    for (size_t i = 0; i < numberOfDimensions; i++) {
        orderOfDimensions[i] = i;
    }
}
```

Takes the number of dims and a pre-allocated array to fill.

Loop from 0 to numberOfDimensions - 1.

Set position i to value i → [0, 1, 2, ...]. This is the default row-major traversal order. ALWAYS call this before setShape for any new tensor.

setShape

```
void setShape(shape_t *shape, size_t *dims, size_t
numberOfDims, size_t *orderOfDims) {
    shape->dimensions = dims;
    shape->numberOfDimensions = numberOfDims;
    shape->orderOfDimensions = orderOfDims;
}
```

Wires all three fields of a shape_t in one call.

Point to the dimensions array (e.g. [3, 4]).

Store the count (e.g. 2 for a 2D matrix).

Point to the order array (e.g. [0, 1]).

setParameterValues

```
void setParameterValues(parameter_t *parameter,
tensor_t *param, tensor_t *grad) {
    parameter->param = param;
    parameter->grad = grad;
}
```

Wires both fields of a parameter_t.

Point to the weight tensor.

Point to the gradient tensor.

13 — setTensorValues / setTensorValuesForConversion

setTensorValues

WHAT IT DOES	Sets all four pointer fields of a tensor_t in one call.
WHY IT EXISTS	Without this you write four separate assignment lines each time you initialise a tensor — easy to forget one.
HOW TO USE IT	Call after allocating all components (data, shape, quant, sparsity) to wire them together.
<pre>void setTensorValues(tensor_t *tensor, uint8_t *data, shape_t *shape, quantization_t *quantization, sparsity_t *sparsity) {</pre>	All four components passed as parameters.
<pre> tensor->data = data;</pre>	Point to the raw byte buffer.
<pre> tensor->shape = shape;</pre>	Point to the shape struct.
<pre> tensor->quantization = quantization;</pre>	Point to the quantization config.
<pre> tensor->sparsity = sparsity;</pre>	Point to sparsity info, or NULL for dense.
<pre>}</pre>	

setTensorValuesForConversion

WHAT IT DOES	Wires output tensor with new data and new quantization, but SHARES shape and sparsity with the original tensor.
WHY IT EXISTS	Type conversion (FLOAT32→SYM_INT32) produces a tensor with the same shape but different data. Sharing shape avoids copying it unnecessarily.
HOW TO USE IT	Used inside conversion functions. Not for general tensor initialisation.
<pre>void setTensorValuesForConversion(uint8_t *data, quantization_t *q, tensor_t *originalTensor, tensor_t *outputTensor) {</pre>	originalTensor provides the shared shape and sparsity.
<pre> outputTensor->data = data;</pre>	New data buffer — different from originalTensor's buffer.
<pre> outputTensor->shape = originalTensor->shape;</pre>	SHARED pointer — both tensors point to the same shape_t. Safe because conversion does not change the shape.
<pre> outputTensor->quantization = q;</pre>	New quantization — the whole point of the conversion.
<pre> outputTensor->sparsity = originalTensor->sparsity;</pre>	SHARED pointer — sparsity pattern does not change during type conversion.
<pre>}</pre>	

14 — transposeTensor

WHAT IT DOES	Swaps two entries in orderOfDimensions. No data moves in memory.
WHY IT EXISTS	Transposing is needed for matrix operations (multiply by B^T). Moving all data would be slow and waste RAM. Swapping metadata is instant and zero-cost.
HOW TO USE IT	Call with (tensor, 0, 1) to transpose a 2D matrix. Call before any operation that needs a transposed view.

<pre>void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index) {</pre>	Takes the tensor and the two dimension indices to swap.
<pre> if (tensor->shape->numberOfDimensions < 2) {</pre> <pre> return;</pre> <pre> }</pre>	Guard: cannot transpose a 1D tensor (a vector has no second dimension to swap). Early exit — nothing to do. No error; 1D is just not transposable.
<pre> size_t temp =</pre> <pre> tensor->shape->orderOfDimensions[dim0Index];</pre>	Classic swap: save dim0's current value in a temporary variable. If you skip this step, you overwrite dim0 and lose its original value.
<pre> tensor->shape->orderOfDimensions[dim0Index] =</pre> <pre> tensor->shape->orderOfDimensions[dim1Index];</pre>	Copy dim1's value into dim0's slot.
<pre> tensor->shape->orderOfDimensions[dim1Index] =</pre> <pre> temp;</pre>	Copy the original dim0 value (saved in temp) into dim1's slot. Swap complete.
<pre>}</pre>	

```
// Before: orderOfDimensions = [0, 1] (row-major)
transposeTensor(tensor, 0, 1);
// After: orderOfDimensions = [1, 0] (column-major)
// data[] bytes: unchanged! Not one byte moved.
```

15 — printTensor / printShape

printTensor

WHAT IT DOES	Reads every element from the tensor's raw byte buffer and prints it, formatted appropriately for each quantization type.
WHY IT EXISTS	Debugging — to inspect actual tensor values during development.
HOW TO USE IT	Call any time you want to verify tensor contents. Remove from production code — printf is slow on STM32.

<pre>void printTensor(tensor_t *t) {</pre>	
<pre> quantization_t *q = t->quantization;</pre>	Local alias for the quantization pointer — used repeatedly below.
<pre> printf("TENSOR BEGIN \n");</pre>	Visual separator — makes it easy to spot where a tensor starts in UART output.
<pre> size_t numValues = calcNumberOfElementsByTensor(t);</pre>	Total number of elements to print.
<pre> int32_t data[numValues];</pre>	VLA (Variable Length Array) — stack-allocated temporary buffer. Size determined at runtime. Used for INT32 case.
<pre> switch (q->type) {</pre>	Choose the print format based on the actual data type.
<pre> case INT32:</pre>	
<pre> printf("INT32Q \n");</pre>	Label so you know which type is printing.
<pre> readBytesAsInt32Array(numValues, t->data, data);</pre>	Convert the raw bytes into an int32_t array in one call.
<pre> for (size_t i = 0; i < numValues; i++) {</pre>	
<pre> printf("%i\n", data[i]);</pre>	%i prints a signed integer.
<pre> }</pre>	
<pre> break;</pre>	Stop here — do not fall through to next case.
<pre> case FLOAT32:</pre>	
<pre> printf("FLOAT32Q \n");</pre>	
<pre> for (size_t i = 0; i < numValues; i++) {</pre>	
<pre> size_t byteIndex = i * sizeof(float);</pre>	Each float is 4 bytes. byteIndex = element index × 4.
<pre> float currentElement =</pre>	
<pre> readBytesAsFloat(&t->data[byteIndex]);</pre>	Read 4 bytes starting at byteIndex and interpret them as a float.
<pre> printf("%f\n", currentElement);</pre>	%f prints a float.
<pre> }</pre>	
<pre> break;</pre>	
<pre> case SYM_INT32: {</pre>	
<pre> symInt32QConfig_t *symQC = q->qConfig;</pre>	Cast qConfig to the correct type to access scale.
<pre> printf("SYM_INT32 \n");</pre>	
<pre> printf("scale=%e\n", symQC->scale);</pre>	Print the scale factor — essential to understand the actual float values.
<pre> for (size_t i = 0; i < numValues; i++) {</pre>	
<pre> size_t byteIndex = i * sizeof(int32_t);</pre>	Each int32 is 4 bytes.

<pre>int32_t currentElement = readBytesAsInt32(&t->data[byteIndex]); printf("%i\n", currentElement); } break; } case BOOL: printf("BOOL\n"); printf("["); for (size_t i = 0; i < numValues; i++) { printf("%d", tensorBoolGet(t, i) ? 1 : 0); if (i + 1 < numValues) { printf(", "); } } printf("]\n"); break; default: printf("WTF\n"); } printf("TENSOR END \n"); printf("_____ \n"); }</pre>	Read 4 bytes as a signed 32-bit integer. Print the quantized integer. Multiply by scale to recover float. Opening bracket for the list. Use tensorBoolGet — NOT t->data[i]. BOOL values are packed 8 per byte. Print comma separator between values, but not after the last one. Closing bracket. Catch-all for unknown types. Informal debug message. Visual separator. Divider line.
---	--

printShape

<pre>void printShape(shape_t *shape) { size_t numberOfDims = shape->numberOfDimensions; printf("NumberOfDims: %lu\n", numberOfDims); printf("Dims: \n"); for (size_t i = 0; i < numberOfDims; i++) { printf("%lu\n", shape->dimensions[i]); } printf("OrderOfDims: \n"); for (size_t i = 0; i < numberOfDims; i++) { printf("%lu\n", shape->orderOfDimensions[i]); } printf("_____ \n"); }</pre>	Local copy of the dimension count. %lu = unsigned long format specifier, matches size_t. Section label. Print each dimension size on its own line. Section label. Print the traversal order. After transpose this will differ from [0,1,2,...]. Divider line.
---	--

16 — copyData / copyShape / copyQuantization / copyTensor

■ **Before calling any copy function:** The destination struct must already be allocated and its pointer fields must point to buffers large enough to receive the data. Copy functions do NOT allocate memory — they only copy bytes.

copyData

<pre>void copyData(tensor_t *dest, tensor_t *src) { size_t numberOfValues = calcNumberOfElementsByShape(dest->shape); size_t sizeData = calcNumberOfBytesForData(src->quantization, numberOfValues); memcpy(dest->data, src->data, sizeData); if (src->sparsity != NULL) { memcpy(dest->sparsity, src->sparsity, sizeof(sparsity_t)); } }</pre>	Use dest's shape to get element count. This is why copyShape must be called first — dest must already have shape set. Calculate exact byte count using src's quantization type. Copy exactly sizeData bytes from src->data to dest->data. Both pointers must be valid and dest must have enough space. Only copy sparsity if src has one. Copying a NULL pointer crashes. Copy the sparsity struct by value. sizeof gives the struct size, not the pointer size.
--	---

copyShape

<pre>void copyShape(shape_t *dest, shape_t *src) { memcpy(dest->dimensions, src->dimensions, src->numberOfDimensions * sizeof(size_t)); memcpy(dest->orderOfDimensions, src->orderOfDimensions, src->numberOfDimensions * sizeof(size_t)); dest->numberOfDimensions = src->numberOfDimensions; }</pre>	Copy the dimensions array. dest->dimensions must already point to an allocated array. How many bytes to copy: number of dimensions × bytes per size_t (4 on STM32). Copy the order array. Same pre-condition. Same size calculation. Copy the count field (not a pointer — direct assignment).
--	--

copyQuantization

<pre>void copyQuantization(quantization_t *dest, quantization_t *src) { switch (src->type) { case FLOAT32: dest->type = FLOAT32; dest->qConfig = NULL; break; case SYM_INT32: </pre>	Branch on source type — different types need different copy logic. Float tensors have no qConfig. Copy the type tag. FLOAT32 has no config struct — set to NULL. Symmetric int32 tensors have a scale + roundingMode config.
---	---

<code>dest->type = SYM_INT32;</code>	Copy the type tag.
<code>symInt32QConfig_t *destQC = dest->qConfig;</code>	Cast <code>dest->qConfig</code> (void*) to the concrete type. <code>dest->qConfig</code> must already point to an allocated <code>symInt32QConfig_t</code> .
<code>symInt32QConfig_t *srcQC = src->qConfig;</code>	Cast <code>src->qConfig</code> to access its fields.
<code>memcpy(destQC, srcQC, sizeof(symInt32QConfig_t));</code>	Deep copy the config struct: <code>scale</code> , <code>roundingMode</code> , <code>qMaxBits</code> . Now <code>dest</code> 's config is independent — changing <code>src</code> 's scale won't affect <code>dest</code> .
<code>break;</code>	
<code>case BOOL:</code>	
<code>dest->type = BOOL;</code>	
<code>dest->qConfig = NULL;</code>	BOOL has no config.
<code>break;</code>	
<code>default:</code>	
<code>PRINT_ERROR("Unknown QType!");</code>	Unknown type — fatal.
<code>exit(1);</code>	
<code>}</code>	
<code>}</code>	

copyTensor — Top-Level Deep Copy

<code>// TODO copy sparsity</code>	Sparsity copying is not yet implemented. The dest tensor keeps whatever sparsity it was initialised with.
<code>void copyTensor(tensor_t *dest, tensor_t *src) {</code>	
<code>copyShape(dest->shape, src->shape);</code>	FIRST: copy shape. <code>copyData</code> needs <code>dest->shape</code> to know the element count.
<code>copyQuantization(dest->quantization, src->quantization);</code>	SECOND: copy quantization. <code>copyData</code> uses <code>src->quantization</code> for byte calculation.
<code>copyData(dest, src);</code>	THIRD: copy data. Depends on both shape and quantization being correct.
<code>}</code>	

★ **Order matters:** `copyShape` → `copyQuantization` → `copyData`. `copyData` calls `calcNumberOfElementsByShape(dest->shape)` — so shape must be copied first. `copyData` also needs `src->quantization` — so quantization must be ready too.

17 — Complete Usage Workflow

[illegible]

18 — Common Mistakes

Direct indexing on BOOL tensor: `data[i]`

■ **Problem:** `data[10]` reads byte 10 (holds 8 mask bits), not bool value 10.

✓ **Fix:** Use `tensorBoolGet(tensor, 10)`.

Using `calcBytesPerElement` for BOOL allocation

■ **Problem:** Returns 1 per element. $1024 \times 1 = 1024$ bytes — 8× too large.

✓ **Fix:** Use `calcNumberOfBytesForData(quant, n)` which applies $(n+7)/8$ for BOOL.

Forgetting `setOrderOfDimsForNewTensor` before `setShape`

■ **Problem:** `orderOfDimensions` contains garbage from uninitialised memory.

✓ **Fix:** Always call `setOrderOfDimsForNewTensor(n, order)` first.

Shallow copy: `tensor_t copy = *original`

■ **Problem:** Copies only the pointers — both share the same data. Modifying one changes both.

✓ **Fix:** Use `copyTensor()` with a fully pre-allocated destination.

Calling `copyTensor` with NULL dest fields

■ **Problem:** `copyData` calls `memcpy(dest->data, ...)` — crashes if `dest->data` is NULL.

✓ **Fix:** Allocate all dest components before calling `copyTensor`.

Calling `tensorBoolSet` on FLOAT32 tensor

■ **Problem:** Guard check triggers `exit(1)` — program crashes.

✓ **Fix:** Only call Bool operations on BOOL-typed tensors.

Transposing a 1D tensor

■ **Problem:** Function returns silently — nothing happens, no error printed.

✓ **Fix:** Transpose requires `numberOfDimensions >= 2`.

Forgetting to call `memset` in `byteConversion` output

■ **Problem:** `byteConversion` calls `memset` internally — but if you reuse an output buffer, stale bits remain.

✓ **Fix:** Always use a freshly zeroed buffer or call `memset` before `byteConversion`.

19 — Quick Reference Card

Function	Returns	What it does	When to call
<code>calcNumberOfElementsByShape(shape)</code>	<code>size_t</code>	Multiply all dims	Before any size calc
<code>calcNumberOfElementsByTensor(tensor)</code>	<code>size_t</code>	Delegates to above	Have <code>tensor_t*</code>
<code>calcNumberOfElementsByParameter(p)</code>	<code>size_t</code>	Uses <code>param->shape</code>	Have <code>parameter_t*</code>
<code>calcBytesPerElement(q)</code>	<code>size_t</code>	Bytes per value (type-dependent)	Pointer arithmetic
<code>calcBitsPerElement(q)</code>	<code>size_t</code>	Bits per value	Sub-byte formats
<code>calcBitsPerTensor(t)</code>	<code>size_t</code>	<code>bits×n</code>	Intermediate calc
<code>calcBytesPerTensor(t)</code>	<code>size_t</code>	<code>bits×n/8</code>	Quick total size
<code>calcNumberOfBytesForData(q,n)</code>	<code>size_t</code>	Exact bytes incl. ceiling	Before <code>reserveMemory</code>
<code>tensorBoolGet(t, idx)</code>	<code>bool</code>	Read one packed bit	Read sparsity mask
<code>tensorBoolSet(t, idx, val)</code>	<code>void</code>	Write one packed bit	Activate/prune weight
<code>getBitmask(s,e)</code>	<code>uint32_t</code>	Byte with 1s at [s,e)	Internal — bit helpers
<code>readByte(data,s,e)</code>	<code>uint8_t</code>	Extract bits to pos 0	Internal — <code>byteConversion</code>
<code>writeByte(existing,data,s,e)</code>	<code>uint8_t</code>	Insert bits at [s,e)	Internal — <code>byteConversion</code>
<code>byteConversion(in,inB,out,outB,n)</code>	<code>void</code>	Repack N values	Internal — conversion fns
<code>getParamFromParameter(p)</code>	<code>tensor_t*</code>	<code>p->param</code>	Get weight tensor
<code>getGradFromParameter(p)</code>	<code>tensor_t*</code>	<code>p->grad</code>	Get gradient tensor
<code>setOrderOfDimsForNewTensor(n,order)</code>	<code>void</code>	Fill [0,1,2,...]	Before <code>setShape</code>
<code>setShape(shape,dims,n,order)</code>	<code>void</code>	Wire shape fields	After allocating shape
<code>setParameterValues(p,param,grad)</code>	<code>void</code>	Wire param pair	After creating both tensors
<code>setTensorValues(t,data,sh,q,sp)</code>	<code>void</code>	Wire all 4 fields	After allocating components
<code>setTensorValuesForConversion(...)</code>	<code>void</code>	Wire output, share shape	Inside conversion fns
<code>transposeTensor(t,d0,d1)</code>	<code>void</code>	Swap order entries	Before transposed matmul
<code>printTensor(t)</code>	<code>void</code>	Print all values	Debugging
<code>printShape(shape)</code>	<code>void</code>	Print dims + order	Debugging shape mismatch
<code>copyData(dest,src)</code>	<code>void</code>	<code>memcpy</code> raw bytes	Part of <code>copyTensor</code>
<code>copyShape(dest,src)</code>	<code>void</code>	<code>memcpy</code> dim arrays	Part of <code>copyTensor</code>
<code>copyQuantization(dest,src)</code>	<code>void</code>	Deep-copy type+config	Part of <code>copyTensor</code>
<code>copyTensor(dest,src)</code>	<code>void</code>	Full deep copy	Save checkpoint